

Week 39: Stochastic Gradient Descent Once More, and Logistic Regression

From minibatches to the first classifier – and from the simple perceptron to automatic differentiation

With pauses for questions

Morten Hjorth-Jensen

Department of Physics and Center for Computing in Science Education,
University of Oslo, Norway

Plan for week 39

Monday lecture (2×45 min):

- ▶ **First 20 min:** stochastic gradient descent once more – the minibatch gradient as an estimate, what an epoch costs, the example where SGD beats gradient descent, the cloud and the schedule (Section 4.7; Project 1 part h)
- ▶ Logistic regression: why a straight line cannot classify, the logistic function and the log-odds, the likelihood and the cross entropy, gradient, Hessian and convexity, gradient descent against Newton, separability and penalties, the softmax (Sections 5.1–5.7)
- ▶ SGD and the adaptive methods on a logistic regression with 50 000 observations (Section 5.9)
- ▶ **Last part:** logistic regression drawn as a single neuron – the simple perceptron – and the same picture read backwards as automatic differentiation in reverse mode (Sections 5.8.1–5.8.2): the stepping stone to neural networks

Tuesday (flipped classroom): your questions, then how to write the report – abstract, introduction, conclusion, references and the declaration of LLM use (weekly exercises; `week39tuesday.ipynb`).

Reading: Chapter 5, Sections 5.1–5.9; Section 4.7 once more. Hastie et al. Section 4.4; Goodfellow et al. Sections 5.7.1 and 8.1–8.3. Deadline for the weekly exercises: **Friday September 25 at midnight.**

Where we were: the cost is a sum, so the gradient is a sum (Section 4.7)

Every cost in this course is a sum over data points, Eq. (4.32),

$$C(\boldsymbol{\theta}) = \sum_{i=1}^n c_i(\mathbf{x}_i, \boldsymbol{\theta}), \quad \nabla C = \sum_{i=1}^n \nabla c_i(\mathbf{x}_i, \boldsymbol{\theta}),$$

and one full gradient costs a pass over all n points for a *single* update. The remedy, Eqs. (4.33)–(4.34): replace the sum by a sum over a random **minibatch** B of M points,

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \gamma_t \mathbf{g}_{B_t}(\boldsymbol{\theta}_t), \quad \mathbf{g}_B = \frac{1}{M} \sum_{i \in B} \nabla c_i \quad (\text{the } 1/M \text{ is the mean convention}).$$

Vocabulary you need on Tuesday and in Project 1

- ▶ **Minibatch**: M rows of the design matrix, drawn without replacement by shuffling the indices once per pass.
- ▶ **Epoch**: one pass through all n/M minibatches; $M = n$ is gradient descent, $M = 1$ is SGD in the strict sense. Everybody says SGD and means minibatches; so do we.
- ▶ **Update count** t : what the learning-rate schedule sees; an epoch is n/M updates.

The minibatch gradient: an unbiased estimate, and what it costs

Drawing M of n points without replacement,
Eq. (4.35):

$$\mathbb{E}[g_B] = \nabla C/n, \quad \text{Var}[g_{B,j}] = \frac{s_j^2}{M} \frac{n-M}{n-1},$$

s_j^2 the variance of the per-point gradients: **unbiased**,
noise $\propto 1/\sqrt{M}$, and exactly zero at $M = n$.

Counting flops, Eq. (4.36): a gradient is two
matrix–vector products,

full gradient $\approx 4np$, minibatch gradient $\approx 4Mp$,

so *one epoch of SGD costs exactly one step of
gradient descent*. The question is never “which is
faster per step” but “which buys more per pass over
the data”.

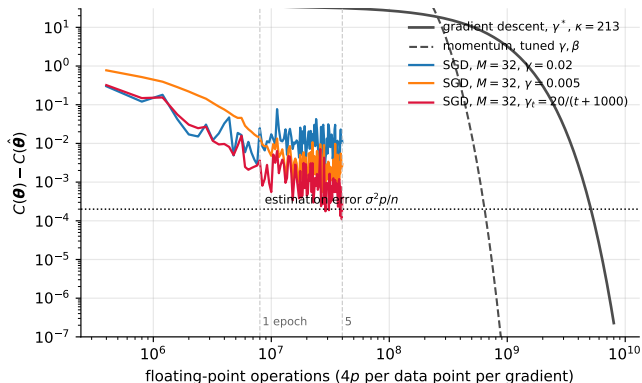
Two regimes, Eq. (4.37)

Gradient descent needs $\sim \kappa \ln(C_0/\epsilon)$ passes to
reach an excess cost ϵ , i.e. $\mathcal{O}(np\kappa \ln n)$ flops to the
statistical floor $\epsilon \sim \sigma^2 p/n$. SGD with a decaying
rate needs $\sim 1/\epsilon$ *updates*, each $4Mp$: $\mathcal{O}(nM/\sigma^2)$
flops, no κ , no $\ln n$. SGD wins by a factor
 $\sim p\kappa \ln n/M$ (Bottou and Bousquet).

Why the floor?

$\hat{\theta}$ is itself an estimate: the next data set moves the
minimum by $\sim \sigma^2 p/n$ in cost (Eq. 2.47).
Optimising below that chases the noise of this
particular sample.

The example where SGD beats gradient descent (book Figure 4.5)



Least squares, $n = 10^5$, $p = 20$ standardised but strongly correlated columns ($\kappa = 213$), unit noise. Per flop: gradient descent at its best γ^* reaches the floor $\sigma^2 p/n = 2 \times 10^{-4}$ after 640 steps (5×10^9 flops); tuned momentum after 81; SGD with $M = 32$ and the schedule $\gamma_t = 20/(t + 1000)$ after about five epochs (4×10^7 flops) – a hundred times less. At a *constant* γ SGD plateaus at 10^{-2} or 3×10^{-3} , well above the floor: that plateau is the subject of the next frame.

The cloud and the schedule (Section 4.7.1)

Near the minimum the iterate is a random walk pulled back by the gradient; along an eigendirection with curvature λ , Eq. (4.39),

$$\text{Var}[e_\infty] = \frac{\gamma \sigma_g^2 / M}{\lambda(2 - \gamma\lambda)} \approx \frac{\gamma \sigma_g^2}{2\lambda M}.$$

At a constant γ the iterate does not converge; it **equilibrates** in a cloud of radius $\propto \sqrt{\gamma/M}$, largest along the flat directions. More iterations do nothing. To shrink it: a smaller γ , a larger M , or – since nothing else is needed at the end – a decaying γ_t , Eq. (4.40),

$$\gamma_t = \frac{t_0}{t + t_1} \quad \text{or} \quad \gamma_t = \gamma_0 \frac{t_0}{t + t_0}.$$

Robbins–Monro: $\sum_t \gamma_t = \infty$ (can still get anywhere) and $\sum_t \gamma_t^2 < \infty$ (finite accumulated noise); $1/t$ does both, a constant does neither.

What we saw last Tuesday

Degree-5 data, $M = 1$, $\gamma = 0.1$: divergence, because a single point can have $2|x_i|^2$ up to 86 – the stability bound holds *per minibatch*. The schedule $(t_0, t_1) = (1, 10)$ froze at 3.0: it decayed too fast for a finite run. $(20, 200)$ reached 0.25; Adam with minibatches 0.14.

Stopping

Not the gradient: the cost on a held-out validation set, stopped when it starts to rise – *early stopping*, a regulariser as much as a termination rule.

The loop you will write for Project 1, part h)

The optimiser knows nothing about data; the data loop knows nothing about the optimiser; the gradient is one function, by hand or by `jax.grad`. Three things that can go wrong, kept apart.

```
def make_batches(n, M, rng):
    idx = rng.permutation(n) # reshuffle every epoch
    return [idx[k:k + M] for k in range(0, n, M)]

theta, state, t = np.zeros(p), {}, 0
for epoch in range(n_epochs):
    for b in make_batches(n, M, rng):
        t += 1
        g = gradient(theta, X[b], y[b]) # same function as the full gradient
        gamma_t = gamma0 * t0 / (t + t0) # Eq. (4.40); a constant is the special case
        theta, state = optimiser_step(method, theta, g, state, t, gamma_t)
    cost_val.append(cost(theta, X_val, y_val)) # for early stopping
```

`optimiser_step` is last week's: "plain", "momentum", "adagrad", "rmsprop", "adam" (Section 4.12). Today we hand it a gradient that is not a linear function of θ for the first time.

Pause and think 1: the same number of flops, three ways

Question

You have $n = 10^5$ points, $p = 20$, and a budget of ten passes over the data. Gradient descent gives you ten steps. With $M = 32$ you get 31 250 updates. Why is the second not simply 3000 times better – what limits it, and what would you set to get the most out of the ten epochs?

Pause and think 1: the same number of flops, three ways

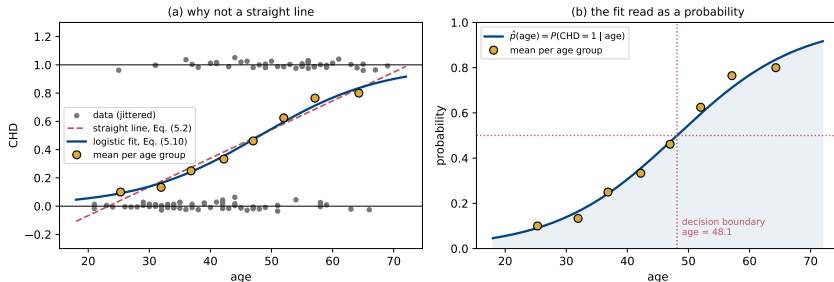
Question

You have $n = 10^5$ points, $p = 20$, and a budget of ten passes over the data. Gradient descent gives you ten steps. With $M = 32$ you get 31 250 updates. Why is the second not simply 3000 times better – what limits it, and what would you set to get the most out of the ten epochs?

Answer

Early on nearly every minibatch step points downhill, so SGD covers in one epoch what gradient descent covers in hundreds (book Figure 4.5). Late in the run the noise takes over: at a constant γ the iterate equilibrates at an excess $\propto \gamma/M$ (Eq. 4.39) and the remaining updates are wasted. So: a rate large enough to descend fast in the first epochs and decaying over the ten (Eq. 4.40, with t_1 of the order of a few epochs' worth of updates, not 10), M a power of two that fills the vector unit, and the stop decided by a validation cost, not by the epoch count. The floor is statistical, $\sim \sigma^2 p/n = 2 \times 10^{-4}$ in the figure; below it the optimiser is polishing noise.

Classification, and why not a straight line (Section 5.1)



Responses $y_i \in \{0, 1\}$: coronary heart disease against age, $n = 100$, 43 with CHD. **(a)** Least squares on the labels, Eq. (5.2), gives $-0.47 + 0.020 \text{ age}$: negative below 23, above one at 73, and a single distant point moves the threshold. What is meaningful is the mean within each age group (yellow): confined to $[0, 1]$ and S-shaped. **(b)** The logistic fit, Eq. (5.10), is that S-shape with two parameters; read as $P(\text{CHD} = 1 \mid \text{age})$ it crosses $1/2$ at 48 years. A *soft* classifier outputs a probability; thresholding it is the hard classifier – the perceptron – as a special case.

The logistic function (Section 5.2)

$$\sigma(t) = \frac{1}{1 + e^{-t}} = \frac{e^t}{1 + e^t} \quad \text{Eq. (5.3)}$$

maps $\mathbb{R}$ onto $(0, 1)$, and three identities do all the work:

$$1 - \sigma(t) = \sigma(-t) \quad (5.4)$$

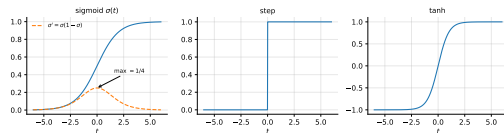
$$\sigma'(t) = \sigma(t)(1 - \sigma(t)) \leq \frac{1}{4} \quad (5.5)$$

$$\text{logit}(p) = \ln \frac{p}{1-p} = \sigma^{-1}(p) \quad (5.6)$$

and $\tanh(t) = 2\sigma(2t) - 1$, Eq. (5.7): the same function, recentred.

Numerics

$1/(1 + e^{-t})$ overflows for $t \ll 0$. Use the two-branch form: for $t \geq 0$ compute $1/(1 + e^{-t})$, for $t < 0$ compute $e^t/(1 + e^t)$ – the same defensive move as the softmax shift of Eq. (5.30).



Book Figure 5.1: σ with its derivative, the step function of the perceptron, and $\tanh$. The derivative peaks at $1/4$ and dies in both tails – remember this when the same function sits inside a deep network.

The model: linear log-odds (Section 5.3)

With the linear predictor $z_i = \mathbf{x}_i^T \boldsymbol{\theta} = \theta_0 + \theta_1 x_{i1} + \dots$, Eqs. (5.8)–(5.10),

$$p(y_i = 1 \mid \mathbf{x}_i, \boldsymbol{\theta}) = \sigma(z_i) = \frac{e^{z_i}}{1 + e^{z_i}}, \quad p(y_i = 0 \mid \mathbf{x}_i, \boldsymbol{\theta}) = 1 - \sigma(z_i),$$

$$\ln \frac{p}{1-p} = \mathbf{x}_i^T \boldsymbol{\theta}$$

What is linear, and what is not

The *probability* is not linear in the features; the *log-odds* are. A unit increase in x_j multiplies the odds $p/(1-p)$ by e^{θ_j} , whatever the other features. This is why logistic regression is the classifier of medicine and economics: every coefficient is an odds ratio.

On the CHD data (week39.ipynb)

In years, $\ln \frac{p}{1-p} = -4.84 + 0.101 \text{ age}$: the odds of CHD multiply by $e^{1.01} = 2.7$ per decade; $p = 1/2$ where the log-odds vanish, $\text{age} = 48.1$. With the age standardised the same fit is $\boldsymbol{\theta} = (-0.37, 1.20)$ – the parameters change, the model does not.

From the likelihood to the cross entropy (Section 5.3)

Each observation is a Bernoulli trial, so, with $p_i = \sigma(\mathbf{x}_i^T \boldsymbol{\theta})$, Eq. (5.11),

$$P(\mathcal{D} \mid \boldsymbol{\theta}) = \prod_{i=1}^n p_i^{y_i} (1 - p_i)^{1-y_i} \quad (\text{one factor handles both } y_i = 0 \text{ and } y_i = 1),$$

and the negative log-likelihood is the **cross entropy**, Eq. (5.13),

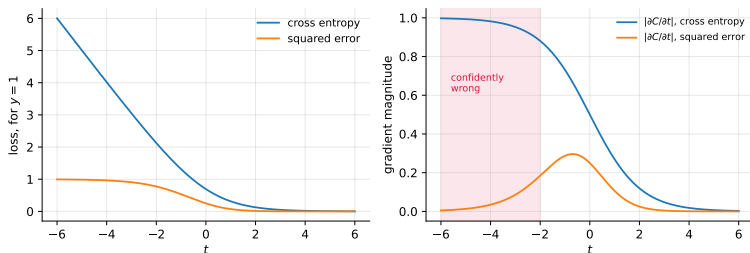
$$C(\boldsymbol{\theta}) = - \sum_{i=1}^n \left[y_i \ln p_i + (1 - y_i) \ln(1 - p_i) \right] = \sum_{i=1}^n \left[\ln(1 + e^{z_i}) - y_i z_i \right] \quad \text{Eq. (5.14),}$$

the second form by $\ln(1 - p_i) = -\ln(1 + e^{z_i})$ and $\ln p_i = z_i - \ln(1 + e^{z_i})$. Use the compact form in code: `np.logaddexp(0, z) - y*z` never takes the log of zero.

The principle, once more

Chapter 3: Gaussian noise $\Rightarrow$ least squares. Here: Bernoulli labels $\Rightarrow$ cross entropy. *Specify the distribution of the target and the loss is determined* – we never chose it.

Why not the squared error on p_i ? (book Figure 5.3)



For $y = 1$, as a function of the linear predictor t . Left: both losses are small when $t \gg 0$; the cross entropy grows *linearly* for $t \ll 0$, the squared error saturates at one. Right: the gradient with respect to t . Cross entropy: $|p - y|$, largest exactly where the model is *confidently wrong*. Squared error: $2|p - y|p(1 - p)$ – multiplied by σ' , which vanishes in the tails: a confidently wrong prediction gets *no* gradient and is never corrected. The same $\sigma' \leq 1/4$ factor returns as the vanishing gradient of deep networks.

Pause and think 2: confidently wrong

Question

Your logistic model gives $p = 0.001$ for a point whose label is $y = 1$. With the cross entropy, how big is the gradient of that point's loss with respect to the linear predictor? And with the squared error $(y - p)^2$? What happens to that point under gradient descent in each case?

Pause and think 2: confidently wrong

Question

Your logistic model gives $p = 0.001$ for a point whose label is $y = 1$. With the cross entropy, how big is the gradient of that point's loss with respect to the linear predictor? And with the squared error $(y - p)^2$? What happens to that point under gradient descent in each case?

Answer

Cross entropy: $\partial c / \partial t = p - y = -0.999$ – a full-size push, Eq. (5.15). Squared error: $\partial c / \partial t = -2(y - p)p(1 - p) = -2 \cdot 0.999 \cdot 0.001 \cdot 0.999 \approx -0.002$, five hundred times smaller, because $\sigma'(t) \approx p$ when p is tiny. Under gradient descent the cross entropy moves that point's predictor at once; the squared error leaves it stranded on the flat tail of the sigmoid – the model is wrong, certain, and unteachable. This is the practical reason the likelihood, and not a least-squares analogy, is the loss for classification (and why a network with sigmoid outputs is trained with the cross entropy).

Gradient, Hessian, convexity (Section 5.4)

Differentiating the compact form (5.14), with $\partial z_i / \partial \theta_j = x_{ij}$ and $\frac{d}{dz} \ln(1 + e^z) = \sigma(z)$, Eqs. (5.15)–(5.17):

$$\frac{\partial C}{\partial \theta_j} = \sum_i (p_i - y_i) x_{ij}, \quad \boxed{\nabla_{\theta} C = -X^T (y - p)}$$

– design matrix against residual, exactly as in least squares (Eq. 4.13) with the prediction replaced by the probability. Once more with $\partial p_i / \partial \theta_k = p_i(1 - p_i)x_{ik}$, Eqs. (5.18)–(5.19):

$$\frac{\partial^2 C}{\partial \theta \partial \theta^T} = X^T W X, \quad W = \text{diag}(p_i(1 - p_i)), \quad 0 < W_{ii} \leq \frac{1}{4}.$$

Convex, hence Chapter 4 applies with its guarantees intact (Eq. 5.20)

$v^T X^T W X v = \sum_i W_{ii} (Xv)_i^2 \geq 0$: the Hessian is positive semi-definite everywhere, every local minimum is global, gradient descent converges for $\gamma < 2/\lambda_{\max}$. The weights vanish for confidently classified points: *the fit is controlled by the points near the decision boundary*. What is lost is the closed form – θ sits inside σ and cannot be extracted.

Optimising the cross entropy: gradient descent and Newton (Section 5.7)

Gradient descent, Eq. (5.31):

$$\theta_{k+1} = \theta_k + \gamma X^T(y - p_k).$$

Stability, Eq. (5.46): the Hessian changes along the path, but at $\theta = 0$ every $W_{ii} = 1/4$ exactly, so

$$\gamma < \frac{8}{\lambda_{\max}(X^T X/n)} \quad (1/n \text{ convention})$$

is the bound that matters: a rate chosen for the flatter optimum overshoots at the start.

Newton–Raphson, Eqs. (5.32)–(5.33):

$$\theta^{\text{new}} = \theta^{\text{old}} + (X^T W X)^{-1} X^T (y - p),$$

recomputing W, p each pass; quadratic convergence, a handful of iterations.

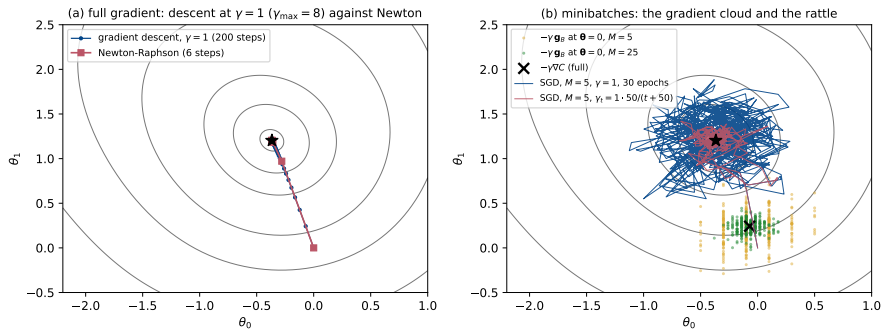
IRLS: a classifier by repeated linear regression (Eq. 5.34)

With the adjusted response $z = X\theta^{\text{old}} + W^{-1}(y - p)$ the Newton step is $\theta^{\text{new}} = (X^T W X)^{-1} X^T W z$ – *weighted least squares*, Eq. (3.14), with weights recomputed every pass: iteratively reweighted least squares.

Cost

Newton: $\mathcal{O}(np^2)$ to form $X^T W X$, $\mathcal{O}(p^3)$ to solve; unbeatable for p in the tens or hundreds (scikit-learn's `lbfgs/newton-cg`). For p in the millions – text with a bag of words – the Hessian is unavailable and SGD with an adaptive method takes over (Section 4.13 drew the line).

Following the steps on the CHD problem (week39.ipynb)



Two parameters, standardised age, so $X^T X / n = I$, $\gamma_{\max} = 8$ and $\kappa(H) = 1.5$ at the optimum. **(a)** Gradient descent at $\gamma = 1$ is below 10^{-4} in excess cost after 22 steps; Newton needs 6 for machine precision, along the same straight path. **Without standardising** – age in years – $\lambda_{\max} = 2122$, $\gamma_{\max} = 3.8 \times 10^{-3}$, $\kappa = 46\,688$: gradient descent is still at 2×10^{-2} after 20 000 steps; Newton still needs 6. **(b)** The minibatch gradients at $\theta = 0$ for $M = 5$ and $M = 25$: centred on the full gradient, spread 0.29 against 0.12 (ratio $2.4 \approx \sqrt{5}$); the stripes for $M = 5$ are the intercept component $\frac{1}{2} - \bar{y}_B$, which takes six values. SGD with $M = 5$ at $\gamma = 1$ rattles at an excess of 1.4×10^{-2} ; with $\gamma_t = 50/(t + 50)$ it is at 8×10^{-5} after 30 epochs.

Pause and think 3: when would you not use Newton?

Question

On the CHD problem Newton converged in six steps and gradient descent needed 22 – both trivial. Name the two situations, one about the data and one about the model, in which Newton's method is *not* the method of choice for logistic regression, and say what replaces it.

Pause and think 3: when would you not use Newton?

Question

On the CHD problem Newton converged in six steps and gradient descent needed 22 – both trivial. Name the two situations, one about the data and one about the model, in which Newton's method is *not* the method of choice for logistic regression, and say what replaces it.

Answer

(1) **Many features:** the Hessian $X^T W X$ costs np^2 to form and p^3 to factorise; with p in the millions (bag-of-words text, one-hot categorical data) it cannot even be stored – minibatch SGD with Adam or RMSProp, at $\mathcal{O}(Mp)$ per update, replaces it. (2) **Separable classes:** then the likelihood has no finite maximiser (Section 5.5), $W \rightarrow 0$ as the weights grow, and the Hessian becomes singular – Newton's step blows up. An ℓ_2 penalty, Eq. (5.23), restores a finite optimum and a positive definite Hessian; early stopping does the same by other means. And a third, for Chapter 8: once the model has hidden layers the cost is no longer convex and a Newton step can head for a saddle.

Code: Newton–Raphson for logistic regression (Section 5.7)

```
def sigmoid(z): # numerically stable, Eq. (5.3)
    out = np.empty_like(z, dtype=float)
    pos, neg = z >= 0, z < 0
    out[pos] = 1.0 / (1.0 + np.exp(-z[pos]))
    ez = np.exp(z[neg]); out[neg] = ez / (1.0 + ez)
    return out

def logreg_newton(X, y, n_iter=25, tol=1e-10, ridge=1e-8):
    n, p = X.shape
    theta = np.zeros(p)
    for it in range(n_iter):
        prob = sigmoid(X @ theta)
        gradient = X.T @ (y - prob) # Eq. (5.17)
        W = prob * (1.0 - prob) # Eq. (5.18)
        H = X.T @ (W[:, None] * X) + ridge * np.eye(p) # Eq. (5.19)
        step = np.linalg.solve(H, gradient) # never invert H
        theta += step
        if np.linalg.norm(step) < tol:
            break
    return theta, it + 1
```

The tiny ridge keeps H invertible on separable data. Gradient descent is the same function with `step = gamma * gradient / n`; the class of Section 5.8 in the book does binary and multiclass with one loop each.

Separability, penalties, and more than two classes (Sections 5.5–5.6)

When the classes are separable, the fit runs away

If some θ classifies every point correctly, then $c\theta$ does too for every $c > 1$ and the likelihood keeps increasing: no finite maximiser, weights $\rightarrow \infty$, probabilities $\rightarrow 0/1$. An ℓ_2 penalty, Eq. (5.23),

$$C_\lambda(\theta) = C(\theta) + \lambda \|\theta\|_2^2,$$

adds 2λ to the Hessian and a finite optimum; ℓ_1 gives sparsity as in Chapter 3. Some penalty is not optional here – it is *necessary*. (scikit-learn penalises by default, $C = 1/\lambda$.)

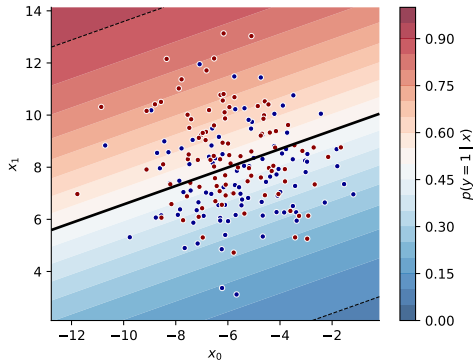
K classes: the softmax, Eqs. (5.24)–(5.29)

One linear predictor per class, $z_{ik} = x_i^T \theta_k$,

$$p(y_i = k | x_i) = \frac{e^{z_{ik}}}{\sum_l e^{z_{il}}},$$

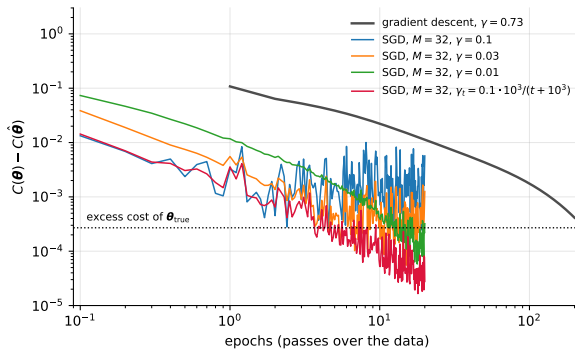
the sigmoid for $K = 2$ (one predictor is redundant; the difference matters). Cross entropy $-\sum_i \sum_k y_{ik} \ln p_{ik}$ with one-hot targets, and the gradient keeps its shape, Eq. (5.29): $\nabla_{\theta_k} C = X^T (p_k - y_k)$. Subtract $\max_l z_{il}$ before exponentiating, Eq. (5.30) – the shift cancels, the overflow does not.

The decision boundary is linear in the features (book Figure 5.2)



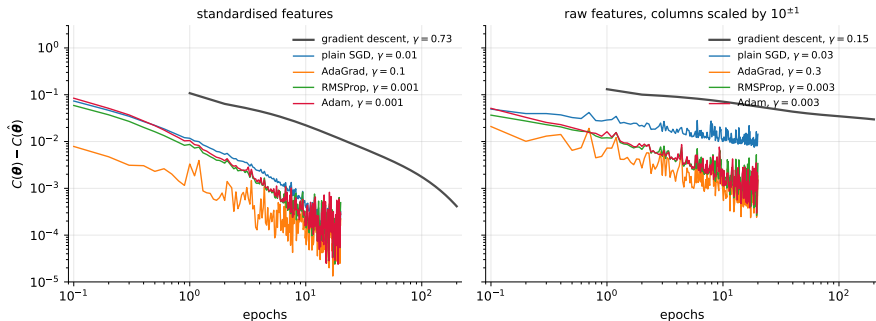
Two features, the fitted probability as contours and the line $x^T \theta = 0$ – *linear* in the features, however non-linear the probability. Non-linear boundaries need non-linear features (Chapter 3, polynomial and other basis functions) or a network (Chapter 8), whose hidden layers learn the features. The points that decide the fit are those near the line: their weights $p(1 - p)$ are the only ones not close to zero.

SGD on a logistic regression with $n = 50\,000$ (Section 5.9, book Figure 5.6)



$p = 20$, correlated standardised features ($\kappa = 173$), labels drawn from a known θ_{true} that classifies only 76.7% correctly. Minibatch gradient, Eq. (5.44): $g_B = X_B^T(p_B - y_B)/M$ – the full-gradient code handed fewer rows. Gradient descent at the bound (5.46), $\gamma = 0.73$: excess 1.1×10^{-1} , 3.9×10^{-2} , 1.1×10^{-2} , 4.1×10^{-4} after 1, 5, 20, 200 epochs. SGD, $M = 32$, $\gamma = 0.01$: 1.8×10^{-3} after 5, 3.2×10^{-4} after 20 – five epochs are worth a hundred, twenty are worth two hundred. The decaying schedule reaches 2.8×10^{-5} . The floor $p/(2n) = 2 \times 10^{-4}$ is the χ^2 statistics of the likelihood ratio: the true parameters sit 2.7×10^{-4} above the minimum. Accuracy saturates first: gradient descent 68.8%, 74.3%, 76.1% after 1, 5, 20 epochs; SGD 76.7% after 20.

AdaGrad, RMSProp and Adam on the same problem (book Figure 5.7, Table 5.1)



Each method at the best of five constant γ 's, $M = 32$. **Left, standardised:** after ten epochs plain SGD (1.9×10^{-4}), AdaGrad (1.3×10^{-4}), RMSProp (2.3×10^{-4}) and Adam (2.2×10^{-4}) are all at the floor, within a factor two – the $\kappa = 173$ comes from *correlation*, which a diagonal preconditioner cannot see. **Right, raw features** (columns scaled by $10^{\pm 1}$, $\kappa = 4.4 \times 10^4$): gradient descent 3×10^{-2} after 200 epochs, plain SGD 1.2×10^{-2} (accuracy 75.9%), the adaptive methods $1\text{--}2 \times 10^{-3}$ (accuracy 76.5%): they absorb the column scales, not the correlation. Their γ is a step length (10^{-3} for Adam/RMSProp); AdaGrad's accumulator shrinks its step as $1/\sqrt{t}$ and needs γ a hundred times larger.

Pause and think 4: standardise, or Adam?

Question

A colleague says: “I never standardise, Adam takes care of the scales.” On the evidence of Table 5.1, what is right about that, what is wrong, and what would you do before training?

Pause and think 4: standardise, or Adam?

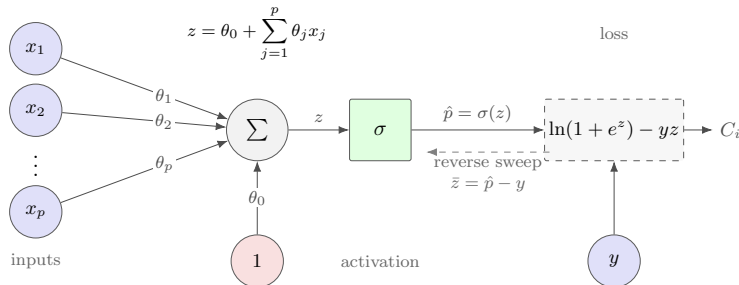
Question

A colleague says: “I never standardise, Adam takes care of the scales.” On the evidence of Table 5.1, what is right about that, what is wrong, and what would you do before training?

Answer

Right: Adam (and RMSProp, AdaGrad) is invariant to the scale of each feature, Section 4.8 – on the raw features it is ten times better than plain SGD and gradient descent is hopeless.
Wrong: it is *not* invariant to correlation between features, and on the raw data the adaptive methods still end a factor five to ten above their own result on standardised data; standardising is free and buys that factor for every method. So: standardise (Section 3.13, inside the cross-validation folds), use minibatches, and reach for Adam when the scales cannot be controlled – in a network the inputs to the inner layers are activations whose scales are set by the training itself, and that is where the adaptive methods earn their keep.

Logistic regression drawn: a single neuron (Section 5.8.1, book Figure 5.4)



Eq. (5.35): $z = \theta_0 + \sum_j \theta_j x_j$, $\hat{p} = \sigma(z)$. Features on the left, a parameter on every edge, the intercept as the weight of an input fixed at one, Σ , σ , and – if we attach the label – the loss $\ln(1 + e^z) - yz$ of Eq. (5.14) as one more node.

The vocabulary of Chapter 8, one week early

x_j : inputs; θ_j : weights; θ_0 : bias; σ : activation function; z : pre-activation. With the step function in place of σ this is Rosenblatt's **simple perceptron** (1958) – the hard classifier of Section 5.1. Logistic regression is a neural network with one neuron and no hidden layer; nothing in this chapter is renamed when we get there.

The perceptron learning rule is the gradient of one observation (Section 5.8.1)

The cost of one observation depends on θ only through z , so by the chain rule through the two nodes, Eq. (5.36),

$$\frac{\partial C_i}{\partial \theta_j} = \underbrace{\frac{\partial C_i}{\partial z}}_{\sigma(z) - y_i = \hat{p}_i - y_i} \cdot \underbrace{\frac{\partial z}{\partial \theta_j}}_{x_{ij}} = (\hat{p}_i - y_i) x_{ij}, \quad \nabla_{\theta} C_i = (\hat{p}_i - y_i) x_i,$$

and a gradient step on that one observation, Eq. (5.37),

$$\theta_j \leftarrow \theta_j - \gamma (\hat{p}_i - y_i) x_{ij}$$

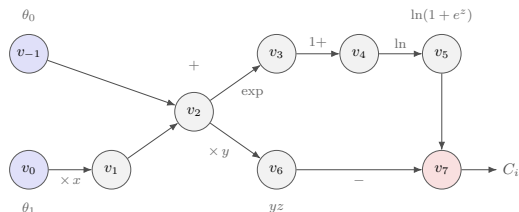
Three names for one formula

Rosenblatt's perceptron rule (hard $\hat{y}$), the *delta rule* of Widrow and Hoff (differentiable output), and SGD with $M = 1$ on the cross entropy, Eq. (5.45): raise the weight of every active input when the output was too small, lower it when too large, in proportion to the error. Summed over i it is $X^T(p - y)$, Eq. (5.17).

Why the sigmoid replaced the step

With the step function the node σ has derivative zero almost everywhere: nothing passes through it, no gradient reaches the parameters, and the perceptron rule needs a separate convergence proof that works only for separable data. With the sigmoid every node is differentiable and the rule is gradient descent on a convex cost – all of Chapter 4 behind it.

The same picture, read backwards: the computational graph (Section 5.8.2)



Book Figure 5.5: the neuron redrawn as the evaluation trace of Section 4.14.2, with the parameters as *inputs*, the data x, y as constants on the edges, and the loss as output. Trace, Eq. (5.38): $v_{-1} = \theta_0$, $v_0 = \theta_1$, $v_1 = v_0 x$, $v_2 = v_1 + v_{-1} = z$, $v_3 = e^{v_2}$, $v_4 = 1 + v_3$, $v_5 = \ln v_4$, $v_6 = y v_2$, $v_7 = v_5 - v_6 = C_i$.

Two readings of one diagram

As a **model**: data flow left to right, parameters are fixed numbers on the edges, a probability comes out. As a **computational graph**: same nodes, same edges, roles exchanged – we differentiate with respect to the parameters, the data are constants. The one structural feature: $z = v_2$ fans out into the softplus branch and the label branch. Every generalised linear model has this shape.

Local derivatives, Eq. (5.39) – eight numbers

$\partial v_1 / \partial v_0 = x$, $\partial v_2 / \partial v_1 = \partial v_2 / \partial v_{-1} = 1$,
 $\partial v_3 / \partial v_2 = v_3$, $\partial v_4 / \partial v_3 = 1$, $\partial v_5 / \partial v_4 = 1/v_4$,
 $\partial v_6 / \partial v_2 = y$, $\partial v_7 / \partial v_5 = 1$, $\partial v_7 / \partial v_6 = -1$.
Everything that follows combines these.

Forward sweep, reverse sweep – with numbers (Eqs. 5.40–5.41)

Take $\theta = (0.5, 0.25)$, $x = 2$, $y = 1$: the tape holds $z = 1$, $v_3 = 2.718$, $v_4 = 3.718$, $v_5 = 1.313$, $C_i = 0.313$, $p = v_3/v_4 = 0.731$.

Forward mode: seed $(\dot{v}_{-1}, \dot{v}_0) = (1, 0)$,
Eq. (5.40)

$\dot{v}_1 = x\dot{v}_0 = 0$, $\dot{v}_2 = 1$, $\dot{v}_3 = v_3$, $\dot{v}_4 = v_3$,
 $\dot{v}_5 = v_3/v_4 = p$, $\dot{v}_6 = y$, $\dot{v}_7 = p - y = -0.269$.
That is $\partial C_i / \partial \theta_0$ – and only that. Seed $(0, 1)$ and sweep
again: every tangent is the one above times x , ending
in $x(p - y) = -0.538$. **Two parameters, two sweeps;**
 p parameters, p sweeps.

Reverse mode: $\bar{v}_7 = 1$, back along the
tape, Eq. (5.41)

$\bar{v}_5 = 1$, $\bar{v}_6 = -1$, $\bar{v}_4 = 1/v_4$, $\bar{v}_3 = 1/v_4$,
 $\bar{v}_2 = \bar{v}_3 v_3 + \bar{v}_6 y = v_3/v_4 - y = p - y$ (the fan-out
sums),
 $\bar{v}_1 = \bar{v}_2$, $\bar{v}_{-1} = \bar{v}_2 = p - y = -0.269$,
 $\bar{v}_0 = \bar{v}_1 x = x(p - y) = -0.538$.
One sweep, both derivatives, read off at the input
nodes.

What the reverse sweep has produced

At v_2 the two branches contribute p and $-y$: the *residual* $\hat{p} - y$, which Section 5.4 obtained by differentiating the log-odds by hand, appears as the sensitivity of the cost to the linear predictor, produced mechanically by the fan-out rule. From v_2 down to the parameters the sweep only multiplies by the input on each edge, 1 and x – the delta rule of the previous frame, done by the machine.

The whole data set, and why every framework runs reverse mode (Eq. 5.42)

The cost is $\sum_i C_i$: n copies of the graph hanging from the *same* p parameter nodes, feeding one summation node.

- **Reverse:** the sum passes $\bar{C}_i = 1$ to every copy, each produces its residual $r_i = p_i - y_i$ at z_i , each parameter node accumulates $\sum_i X_{ij} r_i$:

$$\bar{\theta} = X^T(p - y)$$

in *one* pass, $\mathcal{O}(np)$ – one $X\theta$ forward, one $X^T r$ back. Tape: the n numbers z_i .

- **Forward:** seed e_j , get one component $\sum_i X_{ij} r_i$; p sweeps, $\mathcal{O}(np^2)$.

Measured (book Section 5.8.2, JAX, $n = 5000$)

p	value	reverse	forward
20	0.13 ms	0.19 ms	0.41 ms
200	0.32 ms	0.47 ms	4.53 ms
1000	1.59 ms	3.31 ms	85.9 ms

Reverse: 1.4–2 evaluations of C whatever p – the cheap gradient principle, Prop. 4.2. Forward: 3, 14, 54 evaluations, growing with p . Both agree with $X^T(p - y)$ to 10^{-16} .

The rule

Forward mode: one input direction to all outputs. Reverse mode: one output to all inputs. A cost has one output and p inputs – training asks the reverse-mode question. Price: the tape.

Code: the trace, both sweeps, and JAX (Section 5.8.2)

```
def cross_entropy_one(theta0, theta1, x, y): # the trace of Eq. (5.38), one node per line
    v1 = theta1 * x; v2 = v1 + theta0; v3 = jnp.exp(v2); v4 = 1.0 + v3
    v5 = jnp.log(v4); v6 = y * v2
    return v5 - v6

def reverse_sweep(theta0, theta1, x, y): # Eq. (5.41) by hand
    v1 = theta1 * x; v2 = v1 + theta0; v3 = np.exp(v2); v4 = 1.0 + v3 # forward, onto the tape
    b7 = 1.0; b5, b6 = b7, -b7; b4 = b5 / v4; b3 = b4
    b2 = b3 * v3 + b6 * y # fan-out: two successors, one sum
    return b2, b2 * x # dC/dtheta0, dC/dtheta1

cost_one = lambda th: cross_entropy_one(th[0], th[1], 2.0, 1.0)
theta = jnp.array([0.5, 0.25])
for k in range(2): # forward mode: one sweep per parameter
    _, tangent = jax.jvp(cost_one, (theta,), (jnp.eye(2)[k],))
print(jax.grad(cost_one)(theta)) # reverse mode: one sweep, both -> [-0.269 -0.538]

def cost(theta, X, y): # the whole data set, Eq. (5.43)
    z = X @ theta
    return jnp.mean(jnp.logaddexp(0.0, z) - y * z)
grad_ad = jax.grad(cost)(theta_p, X, y) # == X.T @ (sigmoid(X @ theta_p) - y) / n
```

`jax.grad(cost)` is the gradient of Eq. (5.17): from now on the loss can change (Section 5.10: hinge, exponential, focal) and no derivative is written down again.

Pause and think 5: one layer more

Question

Suppose the inputs x_j of the neuron in Figure 5.4 were not the raw features but the outputs of another layer of such neurons, $x_j = \sigma(\mathbf{w}_j^T \mathbf{u} + b_j)$. What does the reverse sweep do when it reaches $\bar{z} = \hat{p} - y$ – and what is the name of the algorithm?

Pause and think 5: one layer more

Question

Suppose the inputs x_j of the neuron in Figure 5.4 were not the raw features but the outputs of another layer of such neurons, $x_j = \sigma(w_j^T u + b_j)$. What does the reverse sweep do when it reaches $\bar{z} = \hat{p} - y$ – and what is the name of the algorithm?

Answer

It does not stop. From $\bar{z}$ it walks down each edge to the parameter it carries – $\bar{\theta}_j = \bar{z} x_j$, the delta rule – but also to the *input* x_j of that edge: $\bar{x}_j = \bar{z} \theta_j$. Since x_j is itself the output of a neuron, the sweep continues through its activation, $\bar{z}_j = \bar{x}_j \sigma'(z_j)$, and then along its edges to w_j and to the layer before. Layer by layer, the adjoint of each layer's output is multiplied by the transposed weight matrix and by the derivative of the activation: that is **backpropagation**, and Eq. (5.41) is its one-layer special case. The $\sigma' \leq 1/4$ factor is multiplied in once per layer – the reason deep sigmoid networks stop learning, and the reason Chapter 8 will change the activation.

Summary of week 39

- ▶ **SGD once more:** the minibatch gradient is unbiased with noise $\propto 1/\sqrt{M}$ (Eq. 4.35); an epoch costs one full step (Eq. 4.36); per pass over the data SGD wins by $\sim p\kappa \ln n/M$ (Eq. 4.37); at a constant γ it equilibrates in a cloud $\propto \gamma/M$ (Eq. 4.39), so decay γ (Eq. 4.40) and stop on validation error.
- ▶ **Logistic regression:** linear log-odds, Eq. (5.10); Bernoulli likelihood $\Rightarrow$ cross entropy, Eqs. (5.13)–(5.14); gradient $X^T(p - y)$, Hessian $X^T W X$, convex, Eqs. (5.17)–(5.20); Newton = IRLS, Eqs. (5.33)–(5.34); a penalty is necessary when the classes separate; softmax for K classes.
- ▶ **On 50 000 points:** five epochs of SGD are worth a hundred of gradient descent; the adaptive methods add nothing on standardised features and an order of magnitude on raw ones; accuracy saturates long before the cost (Section 5.9).
- ▶ **The neuron:** logistic regression is the simple perceptron with a sigmoid; its gradient is the delta rule, Eq. (5.37); the same diagram read backwards is reverse-mode AD, Eqs. (5.40)–(5.42) – one sweep, all p derivatives, and backpropagation is what happens with one layer more.

Tomorrow, the deadline, and next week

Tuesday: your questions, then *how to write the report* – abstract, introduction, conclusion, references, and the LLM declaration; the weekly exercises are exactly those five pieces of *your* Project 1 report, due **Friday September 25 at midnight**. Project 1 is due October 5. Week 40: neural networks (Chapter 8) – the neuron of today, in layers.